

Spec2INC: Agentic HW/SW Co-Design of UALink-Inspired In-Network Collectives

CHIA Hackathon Proposal | A³ @ MICRO 2026
Roman Kapur

Objective & Scope

UALink 2.0 motivates both collective primitives and block collectives as In-Network Compute (INC) mechanisms. This project will not attempt UALink compliance. Instead, it will build a UALink-inspired RTL/behavioral model that abstracts routing, RAS, security, manageability, and other protocol machinery so the experiment can isolate the architectural tradeoff between fine-grained and block-level collective offload. The goal is to map where each abstraction wins as endpoint count, message size, collective type, and workload characteristics change. Finally we hope to see how the scale-up fabric exposing fine-grained collective primitives versus block-level collective offloads effects workload performance.

CHIA Loop & Method

Spec2INC will be implemented as a CHIA loop rather than a standalone LLM script. ChiaFunction nodes will wrap cocotb/Verilator simulation, synthesis, and workload sweeps. A design agent will receive a read-only architectural contract plus ChiaTools for editing candidate RTL/software and invoking those evaluation nodes. The independent cocotb reference model/checkers will be built as part of the project and remain outside the generated DUT. Counterexamples, cycle counts, traffic metrics, and area/Fmax will feed back to the agent, while CHIA/Ray fans out independent endpoint/message/workload configurations in parallel.



Evaluation & Scaling

- **Compare:** P2P baseline, primitive INC, block-collective INC, and agent-optimized hybrid HW/SW.
- **Sweep:** 2/4/8/16/(32) endpoints, message size, collective type, outstanding work, and fabric parameters.
- **Measure:** completion cycles, bytes/requests, endpoint issue overhead, utilization, area/Fmax, and crossover regions.

Workload-Level Validation

- **Model:** high-level GPU timing for compute, local memory, synchronization, and compute/communication overlap.
- **Drive:** representative AllReduce, ReduceScatter, AllGather, and mixed distributed-AI traces into cycle-accurate INC RTL.
- **Result:** translate block-level gains into end-to-end speedup and test an adaptive primitive-vs-block selection policy.

Software Extension

Beyond isolated microbenchmarks, we would also like to try an adapting representative distributed-AI workloads to each collective interface, retargeting collective calls, scheduling, batching, and primitive-vs-block selection while preserving workload semantics through AI. This will test whether operation-level gains translate into end-to-end speedup across realistic endpoint scales and compute/communication patterns. This will likely involve some artistic modeling of the GPU portion of these workloads, but these will not be completely done in RTL.

Expected Results & Reusable Output

Expected outputs are: (1) an open-source CHIA loop that composes agent, cocotb/Verilator, synthesis, and distributed sweeps; (2) a reusable UALink-inspired INC evaluation harness; (3) verified primitive, block, and hybrid candidate designs; and (4) scaling/crossover maps showing when each HW/SW abstraction is most effective. The main result is the primitive-vs-block design-space boundary and whether a workload-aware, agent-optimized HW/SW policy can improve it - not a claim of UALink protocol compliance.

Compute Budget - \$750

~\$400 for Gemini/agent iterations, ~\$200 for GCP simulation/synthesis sweeps, and ~\$150 contingency for larger endpoint/workload exploration.

References: [1] CHIA Hackathon CFP, A³ @ MICRO 2026. [2] CHIA docs: CHIA Basics, ChiaTool, RISC-V ISA Extension case study. [3] UALink Consortium, UALink Common 2.0 announcement (Apr. 2026). [4] Synopsys, “4 Ways UALink 2.0 Advances AI Scale Up” (collective primitives and block collectives).